FOLLOW-ME WITH RELBOT

In Assignments 1.1 and 1.2, you are asked to code an algorithm to track a person moving in an indoor environment and keep the distance between him/her and the camera installed on the robot. This algorithm then triggers the robot control and makes sure that it keeps a safe distance from the tracked person.

Some suggestions

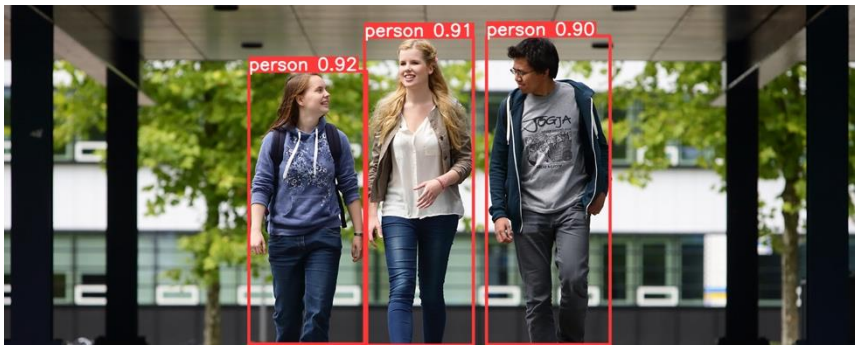
- The object tracking in a video has been discussed in the first assignment of this course. You can use YOLOv8 (or other algorithms) for this task. For the distance estimation, you can use one of the SIDE algorithms.
- A tutorial on how to run a state-of-the-art SIDE (Single Image Depth Estimation) algorithm is reported in depth estimation (kaggle.com). This tutorial demonstrates more than just depth estimation with cars. It first runs YOLOv8 for object detection and then uses MiDaS for inverse depth estimation on the selected car. We expect a similar solution from you! Please note that this algorithm allows you to determine the inverse of the depth value (and non-metric values). Feel free to explore other algorithms or similar solutions.
- You need to "scale" the achieved depths converting them into metric values. To do this, you can use some distances in the scene (such as the height of the tracked person) and the information of the used camera (i.e. focal length and pixel size of the object in the frame).
- The distance is an average value in the bounding box or the silhouette of the tracked person.

PLEASE NOTE: Detecting and tracking a specific person requires configuring the system and recognising the unique identifier provided by the tracking algorithm. Although it can be challenging to maintain a consistent ID, once identified, this ID can be used throughout the sequence. Therefore, it is necessary to develop a simple script at the start to select and lock onto the person's ID.

PLEASE NOTE: In our lectures and in the instructions, we give you some suggestions, but you can find more fit-for-purpose or advanced solutions online. Please, feel free to implement the solutions that you consider more appropriate for the assignment. The development of out-of-the-box ideas is also part of the assignment (and it will also be reflected in the final mark).

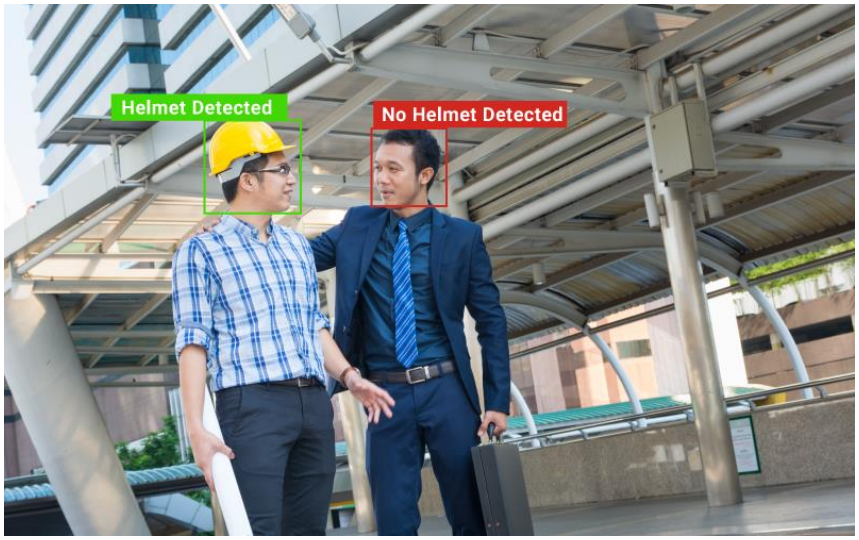
HOW TO FINE-TUNE YOLO FOR HELMET DETECTION

YOLOv8 is a powerful one-stage deep learning framework for object detection and segmentation tasks. It is built on the PyTorch platform and has gained immense popularity due to its ease of use, high performance, and versatility. The pretrained model is using COCO dataset which is different from the Hardhat dataset. Hence, you need to retrain the pretrained (finetuning) model using a sub set of the data from Hardhat (Due to resource restriction to train it on the big dataset) to use it for the hardhat detection.



Pretrained YOLOv8 tested on human detection

Finetune



Retrain the pretrained YOLOv8 and make it work on detecting Hardhats

Links to the materials:

- YOLOv8: https://github.com/ultralytics/ultralytics
- Hardhat dataset: https://doi.org/10.7910/DVN/7CBGOS
- COCO dataset: https://cocodataset.org/
- Tutorial of training custom dataset: https://docs.ultralytics.com/yolov5/tutorials/train_custom_data/